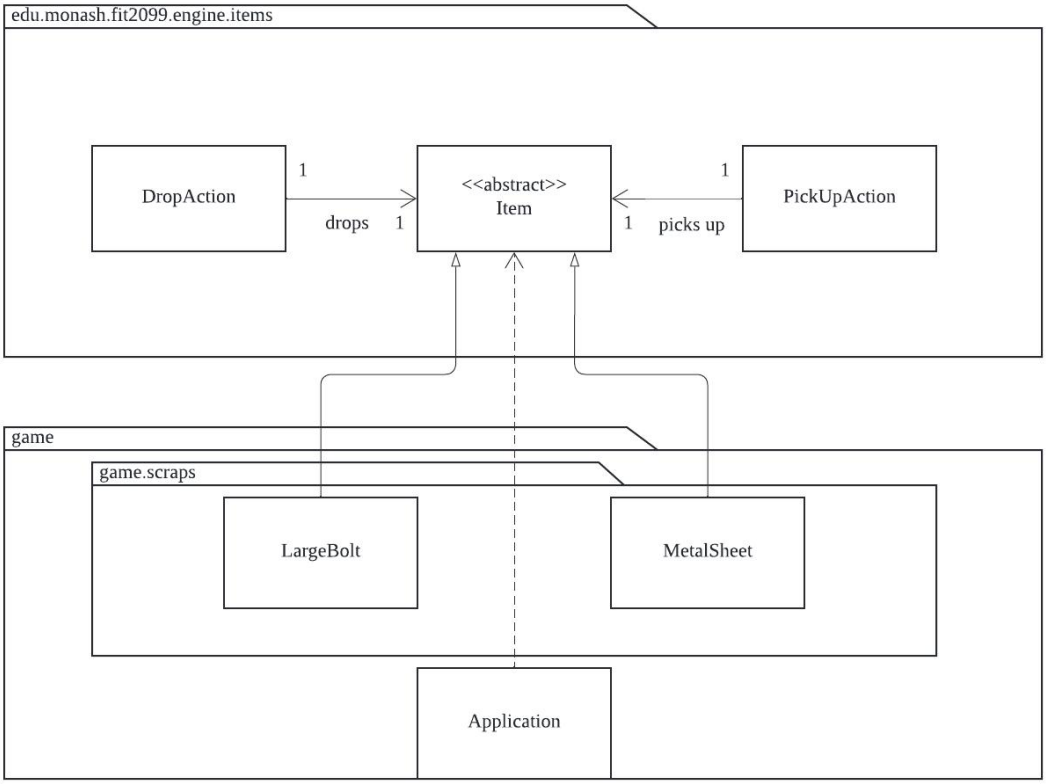


The diagram represents an object-oriented system of Assignment 1 Requirement 1 which aims to create scraps that are able to be picked up using action “Pick Up Action” and be dropped using “Drop Action” by the player.

The object-oriented design principles applied in this design is:

- Single Responsibility Principle
- Dependency Inversion Principle
- Open-Closed Principle

How are the principles being applied?

- Each classes in the system has a very specific functionality. For example, LargeBolt and MetalSheet concrete class each represents a specific type of item in the game. DropAction and PickUpAction concrete class represent a specific type of action that is performed by the player. This approach enhances code readability and maintainability as it clears separation of concern. (Single Responsibility Principle)

- Application class depends on the Item abstract class rather than the concrete implementation (LargeBolt and MetalSheet) with the use of upcasting in code implementation. This promotes flexibility as there are less coupling between the classes due to the reduction of dependencies. (Dependency Inversion Principle)

- The design is open for extension but closed for modification. If it is required to introduce a new type of item, we do not need to modify the Item class which promotes code maintainability.

The pros and cons in this system:

Pros:

- Adhere SOLID principles which promotes code readability, flexibility and maintainability.
- Reduced dependencies and coupling between the classes means reduce the risk of bugs occurring when the code is being modified

Cons:

- May increased the complexity of the code if more classes are introduced during the future.

Let's extend the design with a new class called "Bottles". The responsibility of this new class is to represent a new type of item that can be dropped or picked up by the player in this game. Similar to LargeBolt and MetalSheet concrete class, Bottles concrete class will inherit from the Item abstract class. The actions such as DropAction and PickUpAction give the user the ability to interact these items as a player in the game.

When the Application runs, the menu will prompt and player will be given the option to interact with the item when they are on the same exact coordinate. When the player selects the "pick up" options, it will execute PickUpAction concrete class to pick up different types of items, for example, LargeBolt, MetalSheet and Bottles. When the player selects the "drop" options, it will execute DropAction concrete class to drop the items back to the ground.

Based on the description of the potential extension above, this OOP design allows an easy integration of new features without modifying existing code.